

Aprendizado de Máquina Supervisionado –IMD3002

Aula 24 – Transfer Learning

João Carlos Xavier Júnior

jcxavier@imd.ufrn.br

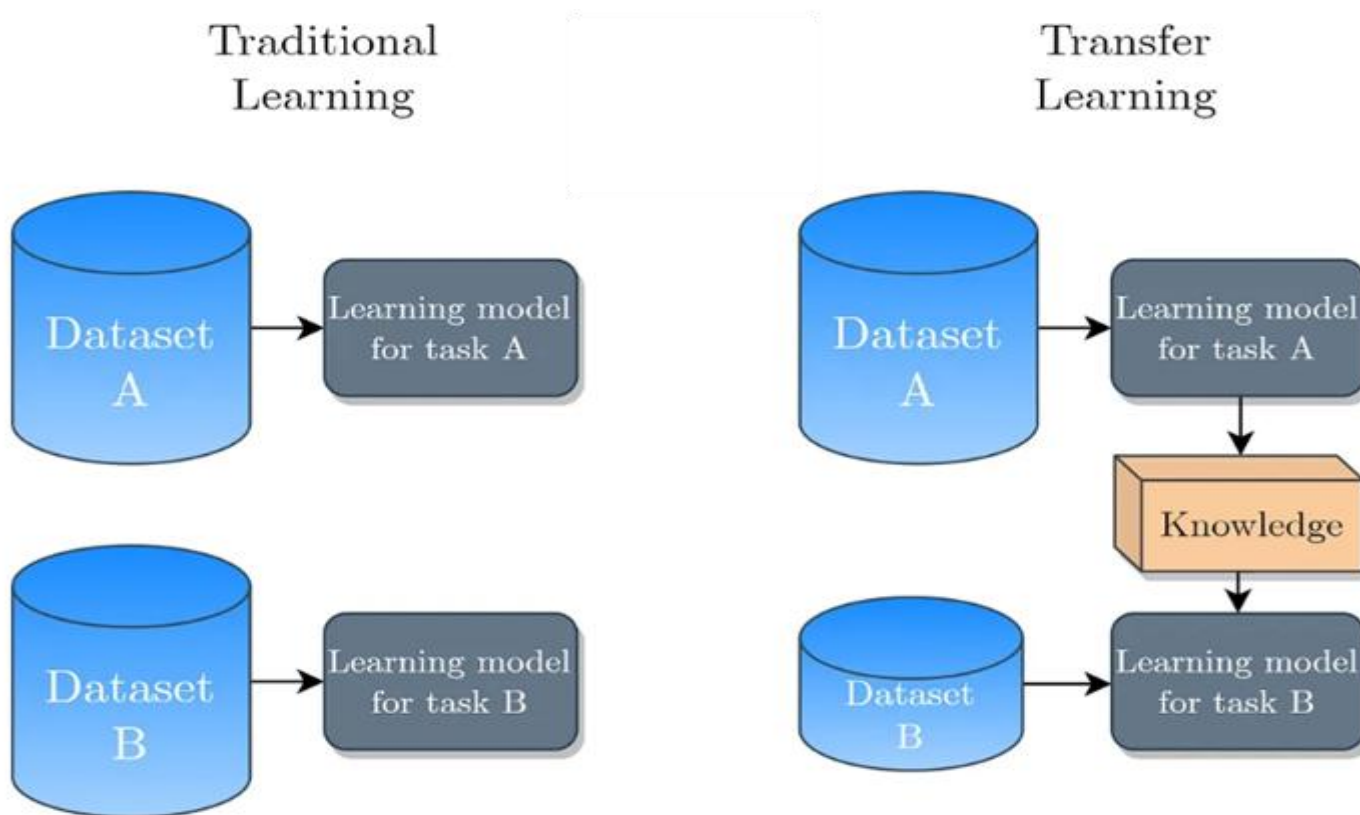
Conceito

❑ O que é **Transfer Learning**?

- ❖ É uma técnica de aprendizado de máquina na qual o conhecimento adquirido em uma tarefa ou conjunto de dados é utilizado outra tarefa relacionada e/ou em um conjunto de dados diferente.

Conceito

❑ O que é **Transfer Learning**?



TL no contexto de Imagens

Imagens

❑ Cachorros e Gatos:



Abyssinian_6



american_bulldog_15



american_pit_bull_terrier_19



Bengal_28

Datasets

MultiLayer Perceptron (MLP) - F1 Score														
#	Base	Treinamento/T este	Conf. 01	Conf. 02	Conf. 03	Conf. 04	Conf. 05	Conf. 06	Conf. 07	Conf. 08	Conf. 09	Conf. 10	Conf. 11	
1	HOG_128_20x20 (900 atributos)	10-fold CV	0,767	0,776	0,771	0,711	0,778	0,741	0,743	0,744	0,743	0,745	0,780	0,754
		70/30	0,725	0,725	0,729	0,745	0,758	0,704	0,717	0,712	0,720	0,720	0,750	0,728
2	HOG_128_24x24 (576 atributos)	10-fold CV	0,747	0,751	0,762	0,750	0,760	0,740	0,744	0,737	0,738	0,744	0,771	0,749
		70/30	0,717	0,729	0,709	0,716	0,742	0,700	0,704	0,700	0,719	0,725	0,725	0,717
3	HOG_128_18x18 (1.296 atributos)	10-fold CV	0,776	0,774	0,762	0,741	0,763	0,736	0,739	0,731	0,745	0,731	0,760	0,751
		70/30	0,725	0,742	0,746	0,366	0,750	0,746	0,738	0,738	0,742	0,725	0,742	0,705
4	HOG_128_30x30 (324 atributos)	10-fold CV	0,734	0,794	0,786	0,772	0,748	0,764	0,763	0,764	0,761	0,756	0,795	0,767
		70/30	0,788	0,788	0,784	0,779	0,779	0,779	0,779	0,779	0,779	0,779	0,779	0,781
5	HOG_64_16x16 (324 atributos)	10-fold CV	0,704	0,733	0,738	0,717	0,715	0,715	0,715	0,716	0,726	0,720	0,746	0,722
		70/30	0,700	0,700	0,709	0,709	0,683	0,696	0,696	0,696	0,688	0,696	0,717	0,699
6	HOG_64_32x32 (36 atributos)	10-fold CV	0,445	0,687	0,710	0,693	0,550	0,709	0,712	0,708	0,705	0,688	0,686	0,663
		70/30	0,512	0,679	0,688	0,674	0,435	0,679	0,663	0,675	0,671	0,684	0,675	0,640
7	HOG_128_30x30 PCA- 75% (34 atributos)	10-fold CV	0,762	0,760	0,759	0,756	0,758	0,764	0,769	0,745	0,766	0,746	0,735	0,756
		70/30	0,725	0,737	0,754	0,742	0,738	0,746	0,725	0,774	0,742	0,736	0,733	0,741
8	HOG_128_32x32 (324 atributos)	10-fold CV	0,746	0,776	0,794	0,765	0,744	0,773	0,768	0,768	0,777	0,761	0,784	0,769
		70/30	0,754	0,763	0,750	0,745	0,742	0,742	0,746	0,742	0,750	0,738	0,746	0,747
9	HOG_256_32x32 (1764 atributos)	10-fold CV	0,791	0,764	0,777	0,716	0,796	0,757	0,754	0,761	0,774	0,754	0,780	0,766
		70/30	0,775	0,771	0,754	0,759	0,779	0,742	0,754	0,754	0,771	0,750	0,754	0,760
10	HOG_128_16x16 (1,764 atributos)	10-fold CV	0,777	0,759	0,740	0,603	0,763	0,717	0,719	0,720	0,740	0,720	0,736	0,727
		70/30	0,746	0,767	0,750	0,721	0,729	0,729	0,717	0,725	0,746	0,729	0,750	0,737
11	HOG_128_16x16 PCA- 75% (115 atributos)	10-fold CV	0,740	0,721	0,740	0,728	0,751	0,757	0,746	0,754	0,760	0,742	0,747	0,744
		70/30	0,700	0,704	0,691	0,696	0,712	0,704	0,692	0,692	0,704	0,692	0,729	0,701
12	LBP_256_6R (50 atributos)	10-fold CV	0,367	0,365	0,339	0,626	0,331	0,455	0,449	0,435	0,480	0,327	0,656	0,439
		70/30	0,409	0,301	0,499	0,675	0,301	0,301	0,300	0,301	0,443	0,301	0,649	0,407
#	Média =>		0,6930	0,7111	0,7184	0,7044	0,6919	0,7040	0,7022	0,7030	0,7163	0,6962	0,7385	
	Desv. Pad. =>		0,1235	0,1209	0,0993	0,0839	0,1396	0,1062	0,1069	0,1085	0,0834	0,1201	0,0390	

Selecionando os Melhores Modelos

Modelos MLP

1	F1_score: 0,788	hidden: 100	activation = relu	solver = sgd	learning = 0,001	max_iter = 500
2	F1_score: 0,788	hidden: (70,70)	activation = relu	solver = adam	learning = 0,0001	max_iter = 2500
3	F1_score: 0,784	hidden: 100	activation = relu	solver = adam	learning = 0,0001	max_iter = 2000
4	F1_score: 0,779	hidden: 50	activation = relu	solver = adam	learning = 0,01	max_iter = 500
5	F1_score: 0,779	hidden: 100	activation = tanh	solver = sgd	learning = 0,001	max_iter = 500
6	F1_score: 0,779	hidden: 120	activation = identity	solver = sgd	learning = 0,01	max_iter = 1500
7	F1_score: 0,779	hidden: 150	activation = logistic	solver = adam	learning = 0,001	max_iter = 1000
8	F1_score: 0,779	hidden: 150	activation = tanh	solver = sgd	learning = 0,01	max_iter = 2000
9	F1_score: 0,779	hidden: (100, 50)	activation = identity	solver = sgd	learning = 0,01	max_iter = 1000
10	F1_score: 0,779	hidden: (100, 50)	activation = logistic	solver = adam	learning = 0,001	max_iter = 1500
11	F1_score: 0,779	hidden: (100, 50)	activation = relu	solver = adam	learning = 0,001	max_iter = 1500

Modelos MLP

❑ Salvando um modelo MLP:

```
import pandas as pd
from numpy import mean
from numpy import std

## Carregando o dataset
dataset = pd.read_csv('/content/HOG_128_30x30.csv', encoding='utf-8')

## Selecionando todos os atributos com exceção da classe
X = dataset.iloc[:, :-1] # Features
y = dataset.classe       # Target variable (classe)

## Carregando o algoritmo
from sklearn.neural_network import MLPClassifier
from sklearn import metrics
from sklearn.metrics import confusion_matrix

## Implementando k-fold CV
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import cross_val_predict
from sklearn.model_selection import KFold
```

Modelos MLP

❑ Salvando um modelo MLP:

```
# 10-fold cv
kf = KFold(n_splits=10, random_state=1, shuffle=True)

## Utilizando uma métrica de tempo
import time

### MLPClassifier e seus Hyperparametros (02)
mlp = MLPClassifier(hidden_layer_sizes=(70,70), activation='relu', solver='adam',
                    learning_rate_init= 0.0001, max_iter=2500)

start_time = time.time()
# Model Accuracy
scores = cross_val_score(mlp, X, y, scoring='accuracy', cv=kf)
end_time = time.time()

# Tempo total de treinamento
execution_time = end_time - start_time
print(f"Tempo de execução para treinamento: {execution_time:.4f} segundos")
print('Accuracy: %.3f (%.3f)' % (mean(scores), std(scores)))
```

Modelos MLP

❑ Salvando um modelo MLP:

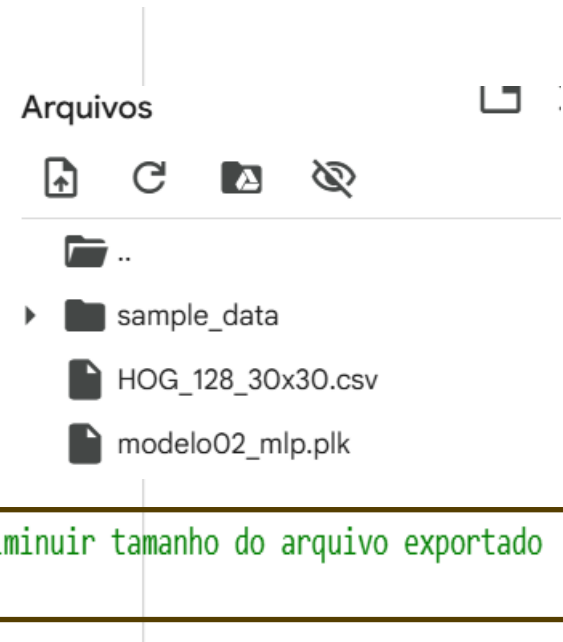
```
# Matriz de confusão p/ kf
y_pred = cross_val_predict(mlp, X, y, cv=kf)
confusion_matrix(y, y_pred)
```

```
# Export do modelo
import joblib
```

```
## pegando modelo treinado
modelo = mlp.fit(X, y)
```

```
# export do modelo - joblib taxa de compressao 3 (0=min, 9=max) para diminuir tamanho do arquivo exportado
joblib.dump(modelo, 'modelo02_mlp.plk', compress=3)
```

Tempo de execução para treinamento: 222.3097 segundos
Accuracy: 0.787 (0.045)



Modelos MLP

❑ Salvando um modelo MLP:

```
# 10-fold CV
kf = KFold(n_splits=10, random_state=1, shuffle=True)

## Utilizando uma métrica de tempo
import time

### MLPClassifier e seus Hyperparametros (11)
mlp = MLPClassifier(hidden_layer_sizes=(100,50), activation='relu', solver='adam',
                    learning_rate_init= 0.001, max_iter=1500)

start_time = time.time()
# Model Accuracy
scores = cross_val_score(mlp, X, y, scoring='accuracy', cv=kf)
end_time = time.time()

# Tempo total de treinamento
execution_time = end_time - start_time
print(f"Tempo de execução para treinamento: {execution_time:.4f} segundos")
print('Accuracy: %.3f (%.3f)' % (mean(scores), std(scores)))
```

Modelos MLP

❑ Salvando um modelo MLP:

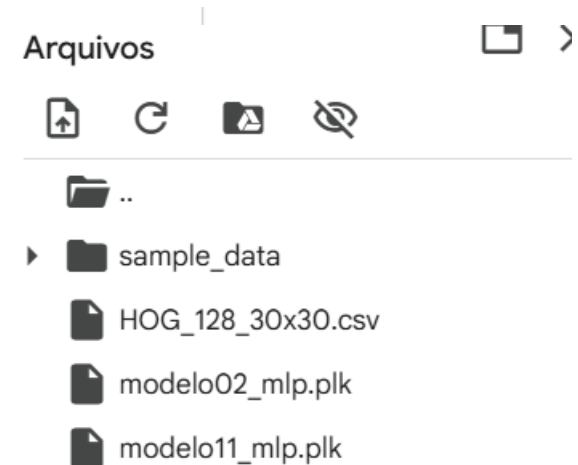
```
# Matriz de confusão p/ kf
y_pred = cross_val_predict(mlp, X, y, cv=kf)
confusion_matrix(y, y_pred)
```

```
# Export do modelo
import joblib
```

```
## pegando modelo treinado
modelo = mlp.fit(X, y)
```

```
# export do modelo - joblib taxa de compressao 3 (0=min, 9=max) para diminuir tamanho do arquivo exportado
joblib.dump(modelo, 'modelo11_mlp.plk', compress=3)
```

```
Tempo de execução para treinamento: 60.2465 segundos
Accuracy: 0.787 (0.048)
['modelo11_mlp.plk']
```



Imagens

❑ Validação do Transfer Learning:



beagle_49



Birman 57



Imagens

❑ Validação do Transfer Learning:

```
hog_features = []
for filename in uploaded_images.keys():
    image = imread(filename)
    image_resized = resize(image, (128, 128))
    fd, hog_image = hog(image_resized, orientations=9, pixels_per_cell=(30, 30),
                        cells_per_block=(2, 2), visualize=True, channel_axis=-1)
    hog_features.append(fd)

# Substituindo as raças de cachorros e gatos por 'dog' e 'cat'
classe = ['dog' if 'beagle' in file else 'cat' if 'Birman' in file
          else file for file in uploaded_images.keys()]

# Salvando csv resultante
df.to_csv('HOG_128_30x30_val.csv', sep=',', index=False)
```

 Birman_82.jpg

 Birman_96.jpg

 HOG_128_30x30_val.csv

 beagle_37.jpg

Modelos MLP

❑ Utilizando um modelo MLP:

```
import pandas as pd

# Export do modelo
import joblib

# Carregamento do modelo exportado
modelo = joblib.load('/content/modelo02_mlp.pkl')

## Carregando o dataset
dataset2 = pd.read_csv('/content/HOG_128_30x30_val.csv', encoding='utf-8')

X2 = dataset2.iloc[:, :-1] # Features

resultados = modelo.predict(X2)
resultados

array(['dog', 'dog', 'dog', 'dog', 'dog', 'dog', 'dog', 'dog', 'dog',
       'dog', 'dog', 'cat', 'dog', 'cat', 'cat', 'cat', 'cat', 'cat',
       'cat', 'cat'], dtype='<U3')
```

LM
classe
dog
dog
dog
dog
dog
dog
dog
dog
dog
dog
cat
cat
cat
cat
cat
cat
cat
cat
cat
cat

Modelos MLP

❑ Utilizando um modelo MLP:

```
import pandas as pd

# Export do modelo
import joblib

# Carregamento do modelo exportado
modelo = joblib.load('/content/modelo11_mlp.plk')

## Carregando o dataset
dataset2 = pd.read_csv('/content/HOG_128_30x30_val.csv', encoding='utf-8')

X2 = dataset2.iloc[:, :-1] # Features

resultados = modelo.predict(X2)
resultados

array(['dog', 'dog', 'dog', 'dog', 'dog', 'cat', 'cat', 'dog', 'dog',
      'dog', 'cat', 'cat', 'dog', 'cat', 'cat', 'cat', 'cat', 'cat',
      'cat', 'cat'], dtype='<U3')
```

LM
classe
dog
dog
dog
dog
dog
dog
dog
dog
dog
dog
cat
cat
cat
cat
cat
cat
cat
cat
cat
cat

Dúvidas ...



Obrigado!!!

